IU2541230781 – CPP71 – PROJECT REPORT

Vehicle Parking & Tracking

Enrollment No.: IU2541230781
Student Name: Hitendra Singh Solanki
Semester: 3
Subject: C++ Guided Project

1. Introduction

Vehicle Parking & Tracking is a C++ console-based application designed to maintain vehicle records and support basic parking operations. It provides a menu-driven interface through which a user can add, display, search, update and delete vehicle records, perform parking transactions, view reports, and save or load data.

The project demonstrates object-oriented programming concepts such as classes and objects, encapsulation, constructors and destructors, inheritance, runtime polymorphism, dynamic memory allocation, file handling, input validation and exception handling. It is intended as an academic demonstration of how a small parking-management workflow can be represented in C++.

2. Problem Statement

Manual parking records can be difficult to maintain, search and update consistently. A simple console-based system is needed to keep vehicle details in a structured form, support parking entry and exit operations, and retain records between application sessions.

This project addresses that need through menu-based record management, parking transactions, reporting and file-based persistence. It is designed for educational use and is not a complete commercial parking solution.

3. Objectives

- Maintain vehicle records through add, display, search, update and delete operations.
- Support parking operations, including parking a registered vehicle and processing its exit.
- Demonstrate object-oriented design through separate classes and class relationships.
- Use dynamic memory allocation and release it correctly.

• Save records to a data file and load existing records when the program starts.

• Provide reports and validate user input to reduce common data-entry errors.

• Prepare documentation, UML diagrams, screenshots and test evidence for academic submission.

4. Functional Requirements

1. Add Record: accept vehicle number, vehicle type and owner details required by the program.

2. Display Records: show vehicle records currently stored in the application.

3. Search Record: locate a record using the supported search field.

4. Update Record: modify details of an existing record.

5. Delete Record: remove a selected record after validation checks.

6. Main Transaction / Process: provide parking, vehicle-exit and active-parking operations.

7. Report: display a summary of available records and parking activity.

8. Save / Load: provide access to file persistence operations.

9. Exit: close the application and save updated data as implemented by the program.

These requirements follow the supplied CPP71 blueprint. Check the final source code before claiming any specific behaviour in the final report.

5. System / Project Flow

The overall flow is: User → Main Menu → ParkingSystem → Entity Classes / Parking Records → Main Transaction or Report → FileManager → Data File.

At startup, the application loads saved records. The user selects an operation from the main menu. The system validates the input, performs the requested record-management or parking operation, and updates in-memory data. Data is saved through the file manager, including during exit according to the implementation.

The blueprint recommends keeping main.cpp focused on creating the system object, showing the menu and calling system functions. This project uses a single source file, so the report should describe the implementation actually submitted.

6. Class Descriptions

The class summary below follows the supplied blueprint and the described program design. Confirm member names and function names against the final source code before submission.

Class	Data Members	Member Functions	Responsibility
Person	Common person information	name / shared person details	virtual display/details function
Owner	Stores owner details	owner ID, name, phone (as implemented)	display/details override
Vehicle	Stores vehicle information	vehicle number, type, owner name	constructor, display
Slot	Represents a parking slot	slot ID, type, status	reserve(), release()
ParkingRecord	Represents one parking transaction	record ID, vehicle number, slot ID, entry time, exit time, fee	calculateFee(), closeRecord()
ParkingSystem	Coordinates the parking application	vehicle, slot and parking-record arrays/pointers	parkVehicle(), exitVehicle(), report(), record-management operations
FileManager	Manages data persistence	file name / file streams	save(), load()

7. OOP Concepts Used

Classes and Objects: classes represent the system and entities used by the parking workflow; objects hold data and provide operations.

Encapsulation: data and functions that operate on it are grouped within classes, with access controlled through the class interface.

Constructors: constructors initialise objects when they are created. The design includes default and parameterised construction where implemented.

Destructor: the ParkingSystem destructor releases dynamically allocated arrays using delete[].

Inheritance: Owner derives from Person in the described implementation, reusing common person details.

Runtime Polymorphism: a virtual display/details function can be overridden by Owner and invoked through a base-class interface.

Dynamic Memory: arrays are allocated dynamically with new[] and released with delete[] to manage entity capacity.

File Handling: file streams store and retrieve application records in a text file.

Validation and Exception Handling: inputs are checked and invalid operations are handled to improve reliability. Describe only the checks and exceptions actually present in the final code.

8. File Handling and Data Persistence

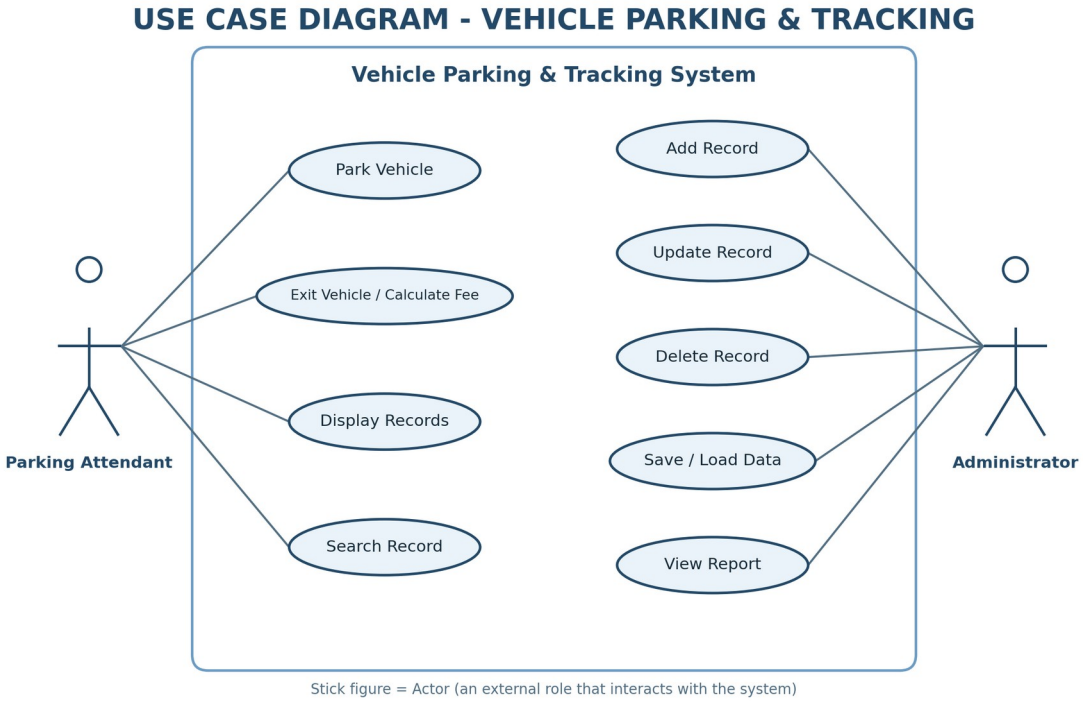
The program uses file handling to preserve data between runs. At startup, saved data is loaded from the configured data file. During use, the program updates its in-memory records; save operations and program exit write current data back to the file.

To verify persistence, add a test record, exit through the program's Exit option, restart the application and check whether the record is still present. The data file is created in the program's current working directory unless the source code specifies another path.

The described version uses parking_data.txt. Keep this file with the submitted project when it contains records that need to be retained, and do not manually edit it unless the format is understood.

9. UML Diagrams

Figure 1. Use Case Diagram



The stick-figure symbol represents a UML Actor: an external role that interacts with the system. In this diagram, the actors are the Parking Attendant and Administrator.

Figure 2. Class Diagram

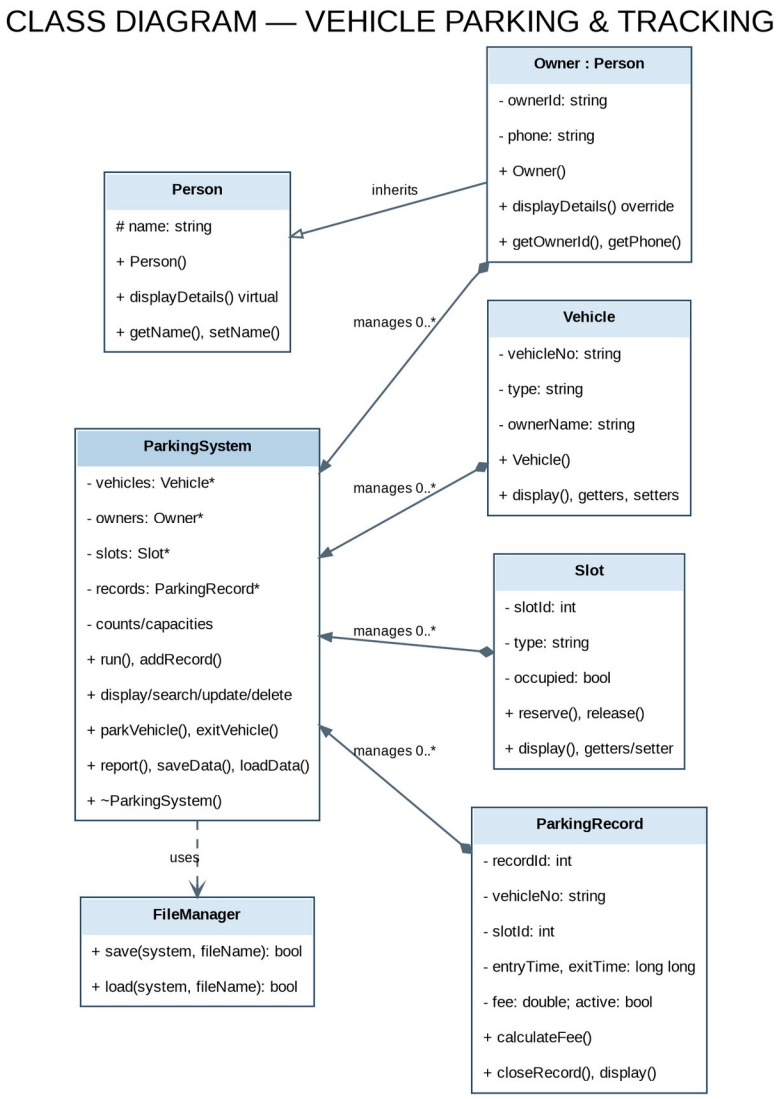


Figure 3. Sequence Diagram 1 - Main Parking Transaction

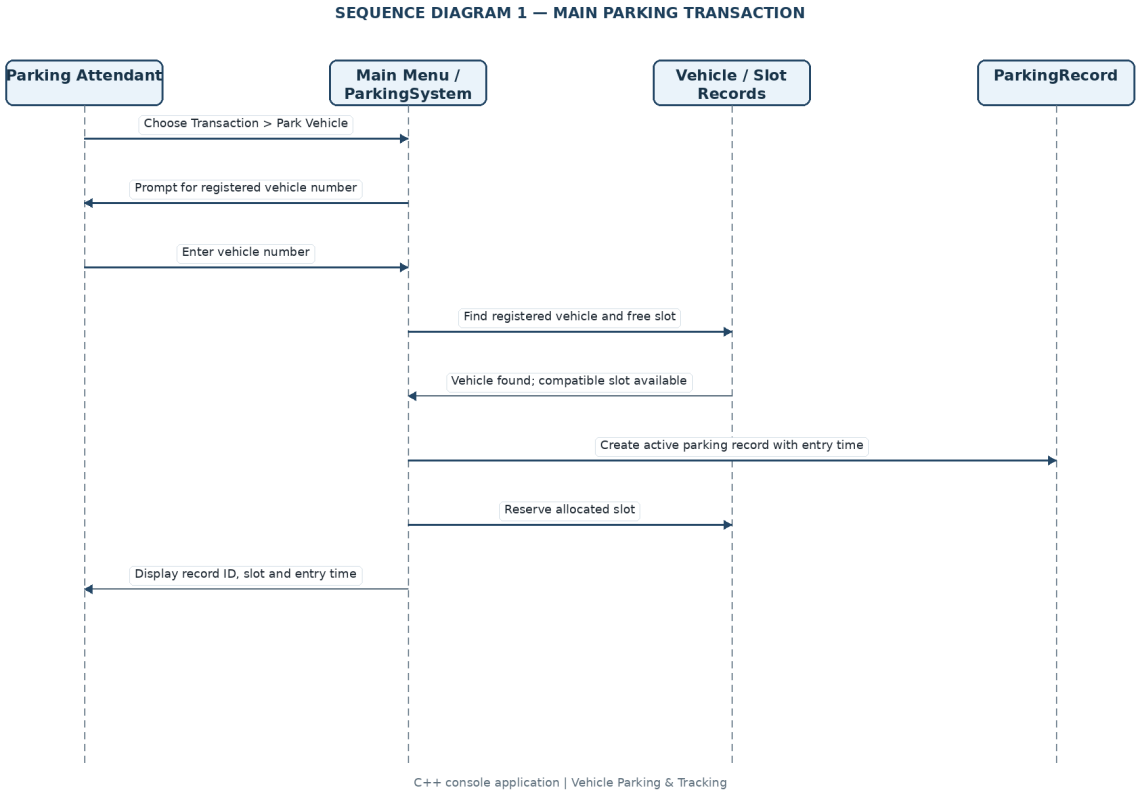
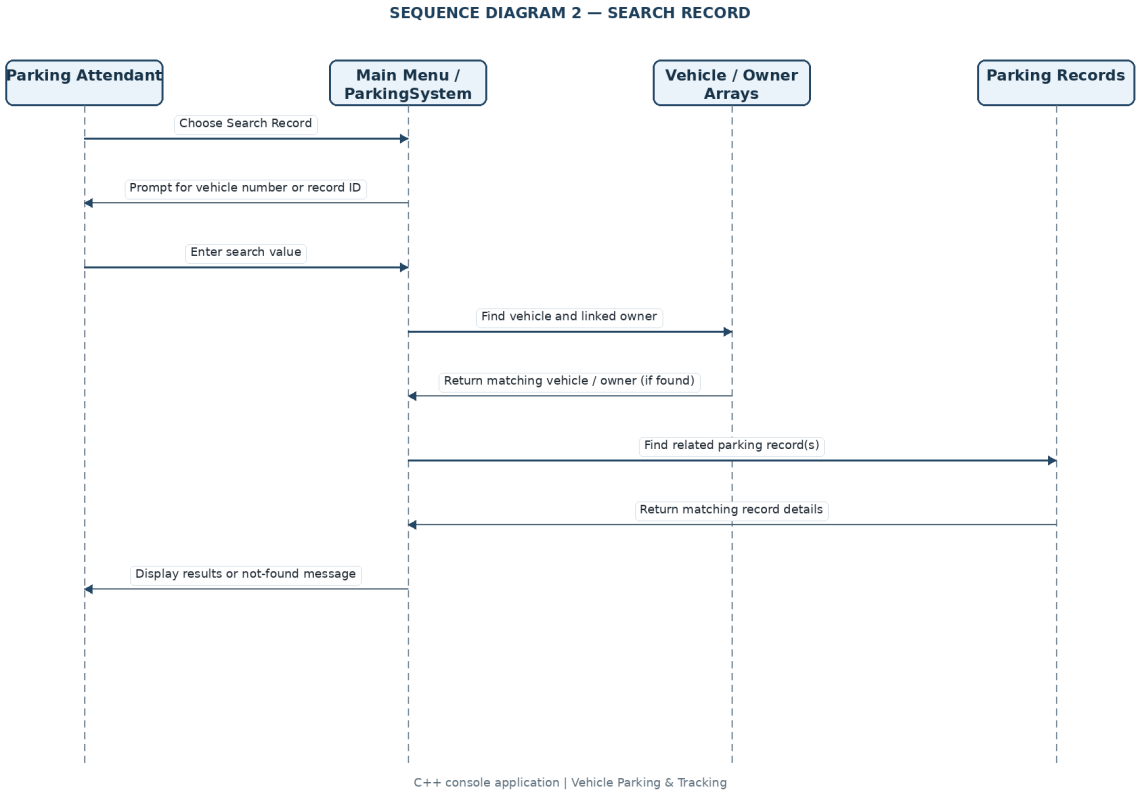


Figure 4. Sequence Diagram 2 - Search Record



10. Implementation Details

The application is implemented in C++ and provides a text-based menu. The main menu exposes record management, parking transactions, reports, save/load operations and exit.

The described implementation uses a Person base class and an Owner derived class, together with Vehicle, Slot, ParkingRecord, ParkingSystem and FileManager classes. ParkingSystem coordinates operations, while FileManager is responsible for loading and saving data.

The implementation uses dynamic arrays and file streams. The program should be compiled with a C++17-compatible compiler. The executable name and compilation command must match the files included in the final project folder.

Source file: main.cpp (single-file implementation). Suggested build command: g++ -std=c++17 main.cpp -o VehicleParking.exe. Run in Windows PowerShell with .\VehicleParking.exe. Use this command only if the source file and compiler are in the stated environment.

11. Testing and Results

The student reports that all 15 test cases were executed and passed. The Actual column records the observed successful outcome for each test.

Test ID	Input / Operation	Expected	Actual	Status
T01	Add Record — valid vehicle details	Record is added and appears in records list	Record added successfully and appeared in the records list.	Pass
T02	Display Records — one or more records exist	Stored records are displayed	Stored vehicle records were displayed.	Pass
T03	Search Record — search for an existing vehicle	Matching record is shown	The matching vehicle record was displayed.	Pass
T04	Search Record — search for unknown vehicle number	A not-found message is displayed	A not-found message was displayed for the unknown vehicle number.	Pass
T05	Update Record — update an existing record	Updated details are displayed when searched/viewed	Updated details were displayed when the record was viewed.	Pass
T06	Delete Record — delete an existing record	Record no longer appears in the records list	The deleted record no longer appeared in the records list.	Pass
T07	Park Vehicle — registered vehicle and available slot	Parking operation starts and slot is assigned	Parking started successfully and a slot was assigned.	Pass
T08	Park Vehicle — vehicle not registered	Operation is rejected with a suitable message	The system rejected the unregistered vehicle and displayed a suitable message.	Pass
T09	Exit Vehicle — vehicle currently parked	Exit is processed and fee is displayed	The exit was processed and the parking fee was displayed.	Pass
T10	Active Parking — view active parking entries	Currently active parking records are displayed	Active parking records were displayed.	Pass
T11	Report — select report menu option	Parking/record summary is displayed	The report summary was displayed.	Pass
T12	Save Data — invoke save operation	Current records are written to the data file	The current records were saved to the data file.	Pass
T13	Load Data — invoke load operation	Saved records are loaded into the application	Saved records were loaded into the application.	Pass
T14	Persistence — save, exit, restart, then display records	Previously saved record is still available	The previously saved record was available after restarting the application.	Pass
T15	Validation — enter an invalid menu/input value	Invalid input is handled without an unexpected crash	Invalid input was handled without an unexpected crash.	Pass

12. Sample Screens / Output

Insert screenshots captured from your own running program here. Recommended screenshots: main menu, adding/displaying a vehicle record, parking transaction, report screen, and the persistence test after restarting the application.

13. Limitations

The application is console-based and does not provide a graphical user interface or mobile/web access.

The data file is a simple local file; it does not provide database-level concurrency, user authentication or advanced backup facilities.

The program is designed for a small academic demonstration and may need additional validation, stronger error recovery and more extensive testing before real-world use.

Parking charges and slot capacity are based on the values implemented in the source code and may not match a real parking facility.

14. Future Scope

A graphical user interface could make the system easier to operate.

A database could replace the text file for more scalable and reliable storage.

User authentication, roles and audit logs could be added.

Reports could include date-wise income, occupancy and vehicle history.

Online booking, QR-code tickets, notifications and payment integration could be added in a future version.

15. Conclusion

Vehicle Parking & Tracking provides an academic example of using C++ to organise vehicle records and parking operations through a menu-driven console application. It brings together object-oriented programming, dynamic memory and file handling in one project.

The project is ready for submission only after the final source code is checked, all four UML diagrams are inserted, 15 test cases are completed using actual observed results, screenshots are added, and the required GitHub repository link is entered in the submission materials.